AIDR-Cloud: An Agentic AI–Driven Incident Response Framework for Cloud Failures

Akshay Pratinav
Intuit Inc
Mountain View, USA
akshay_pratinav@intuit.com

Archana Kataria
Intuit Inc
Mountain View, USA
archana_kataria@intuit.com

***Abstract*— Modern cloud-native systems comprise hundreds of microservices, multi-region deployments, and streaming data pipelines. This complexity has dramatically increased the frequency, impact, and opacity of operational failures. Traditional incident response, which is built around manual troubleshooting, static rules, and prewritten runbooks, struggles to meet today's latency and reliability requirements. This paper presents AIDR-Cloud, an agentic AI–driven incident detection and response framework that combines real-time telemetry, a domain-specific cloud knowledge graph, and multi-agent large language model (LLM) reasoning to autonomously diagnose [11] and remediate failures in cloud environments. AIDR-Cloud introduces a three-layer architecture: a Perception Layer that continuously ingests metrics, logs and traces; a Cognition Layer composed of specialized LLM agents that perform hypothesis generation, validation and remediation planning over a knowledge graph of cloud resources and dependencies; and an Action Layer that executes guarded remediation operations via Kubernetes and cloud-provider APIs under a safety and governance framework. We evaluate AIDR-Cloud along three complementary axes. First, in controlled fault-injection experiments on multi-region Kubernetes and Kafka workloads, we compare AIDR-Cloud against experienced human operators using standard observability and runbook tooling. Second, we analyze the cost and latency overheads of AIDR-Cloud by decomposing end-to-end response time into perception, cognition and action components, and by estimating LLM token usage, control-plane calls and infrastructure cost as a function of incident rate. Third, we report outcomes from a multi-week deployment of AIDR-Cloud on real production incidents, including changes in MTTR and escalation behavior, as well as qualitative improvements in postmortem quality and runbook evolution. Across both synthetic and real incidents, AIDR-Cloud reduces MTTR relative to the human baseline, achieves faster root-cause analysis, reduces the rate of human escalation, and selects correct remediation actions in a large fraction of cases, while adding only a small amount of reasoning latency and no unsafe actions due to strict guardrails. These results show that agentic AI can act as a statistically well-characterized and cost-conscious SRE co-pilot, significantly improving reliability and operational efficiency in complex cloud systems.**

Keywords—Incident Response, Agentic AI, Cloud Reliability, Distributed Systems, Disaster Recovery, LLM Agents

I. INTRODUCTION

Large-scale cloud platforms now operate at levels of concurrency, heterogeneity and dynamism that exceed human cognitive limits. Applications span multi-region Kubernetes clusters, managed data stores, streaming systems such as Kafka, and diverse control planes. Failures may propagate across layers including networking, orchestration, services, and data pipelines, so that seemingly minor anomalies rapidly escalate into customer-visible outages [1].

Despite advances in observability, incident response remains heavily human-centric and fragile. Operators are still responsible for manually correlating metrics, logs and traces across disparate dashboards. Predefined runbooks are often static, difficult to maintain, and quickly become outdated as architectures evolve. Escalations depend on tribal knowledge and tacit expertise accumulated over years. As systems grow in complexity, these approaches fail to scale with the combinatorial explosion of possible failure chains and the sheer volume of telemetry. The resulting detection and diagnosis delays lead to prolonged outages and inconsistent remediation quality, while also increasing operational toil and on-call fatigue.

At the same time, agentic AI [2] has emerged as a promising paradigm for complex operations. In agentic systems, LLMs are used not only for language generation but also as reasoning [12][13] and planning engines that can call tools [14], maintain memory, and critique their own outputs. Unlike traditional AIOps, which focuses primarily on anomaly detection and pattern mining, agentic systems can formulate and test hypotheses, plan multi-step actions over APIs and tools, and reflect on and revise their reasoning. These properties align closely with the cognitive demands of incident response, where operators must rapidly synthesize diverse signals, reason under uncertainty, and coordinate complex remediation actions.

In this paper, we explore how agentic AI can be applied to cloud incident response and disaster recovery. We introduce AIDR-Cloud, an agentic incident response framework that unifies perception, cognition and action by integrating real-time telemetry, LLM-based reasoning, and safe remediation operations over Kubernetes and cloud-provider APIs. The cloud environment is modeled as a knowledge graph [17] that captures dependencies among services, regions, meshes, gateways, data stores and streaming components, providing a structured substrate for causal reasoning [16]. AIDR-Cloud implements a multi-agent root-cause analysis loop in which specialized LLM agents, including Detector, Hypothesis, Validator, Planner and Critic agents, collaborate to generate, test and refine explanations and remediation plans. A practical safety and governance framework constrains action execution through policies, canary rollouts, auto-rollback mechanisms and human-approval modes for high-risk changes.

We evaluate AIDR-Cloud on a multi-region Kubernetes and Kafka environment subjected to realistic failures such as node outages, control-plane degradation, gateway under-scaling, DynamoDB throttling and cross-region latency spikes. Across these scenarios, AIDR-Cloud substantially

SoutheastCon 2026 | 979-8-3315-4642-7/26/$31.00 ©2026 IEEE | DOI: 10.1109/SOUTHEASTCON63549.2026.11476571

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:36:52 UTC from IEEE Xplore. Restrictions apply.

reduces MTTR, accelerates root-cause analysis, and decreases the need for human escalation. By positioning AIDR-Cloud as an agentic SRE co-pilot, rather than a purely advisory AIOps tool [6], we demonstrate that LLM agents can meaningfully augment and, in well-guarded situations, partially automate incident response in production-grade environments.

II. BACKGROUND AND RELATED WORK

Cloud reliability engineering has evolved to address the challenges of large-scale distributed systems. Practices such as site reliability engineering (SRE) formalize the use of error budgets, service level objectives (SLOs), and structured incident management processes. Cloud platforms employ mechanisms such as autoscaling, health checks, circuit breakers and load balancing to maintain availability. However, real-world incidents frequently involve coupled failures that span multiple subsystems. For example, regional control-plane degradation may interact with gateway saturation and data store throttling, leading to cascading impacts that cross the boundaries of individual services or components. While these mechanisms mitigate many issues, they do not eliminate complex failure chains.

Traditional incident response typically follows a pipeline of alerting, triage, diagnosis, and remediation. In practice, this pipeline is burdened by several factors. During major incidents, operators face signal overload, where thousands of alerts and events are triggered, many of them redundant or indirectly related. Telemetry is often siloed, with metrics, logs and traces stored and visualized in separate systems, requiring operators to mentally correlate information across different tools. Runbooks attempt to encode known failure modes and remediation steps, but they are inherently static and difficult to maintain as architectures, dependencies and services evolve. As a consequence, real-world incident response relies heavily on a small number of experts, leading to human bottlenecks, hero culture, and significant cognitive fatigue during prolonged incidents.

AIOps systems introduced machine learning and statistical techniques into cloud operations. These systems can detect anomalies, cluster alerts, and sometimes correlate events based on historical patterns. Despite their value, many existing AIOps tools focus primarily on symptom detection rather than end-to-end causal reasoning and remediation. They often act as recommendation engines that suggest potential actions but do not execute them directly. In addition, many lack policy-aware planning capabilities or explicit safety constraints and are deployed as point tools rather than as integrated actors within incident workflows. Human operators must still interpret the outputs and decide on concrete remediation actions.

Agentic AI systems extend beyond these capabilities by using LLMs as autonomous agents [3]. Such systems can plan multi-step workflows, use external tools and APIs, maintain explicit working memory, and critique or refine their outputs. These features are particularly relevant to incident response, where one must quickly gather evidence, propose hypotheses, test them against data, and execute multi-step mitigation plans. AIDR-Cloud brings these ideas to the cloud reliability domain by embedding LLM agents into a structured incident pipeline, grounded in a domain-specific knowledge graph and a guarded action surface that ensures safe interaction with the underlying infrastructure.

TABLE I. QUALITATIVE COMPARISON WITH TRADITIONAL AND AIOPS APPROACHES

Capability	***Traditional Runbooks***	***AIOps Monitoring Tools***	***AIDR-Cloud***
Anomaly detection	Manual thresholds	Yes	Yes
Causal reasoning over dependency	Manual mental model	Limited / indirect	Explicit via knowledge graph
Automated RCA	No	Partial	Yes
Automated remediation execution	No	Rare/Limited	Yes (guarded actions)
Closed-loop feedback from actions	Informal	Limited	Yes

Table I qualitatively compares AIDR-Cloud with traditional runbook-driven operations and existing AIOps monitoring tools. While conventional approaches rely on human operators to perform anomaly correlation, causal reasoning and remediation, AIOps tools primarily assist with detection and alert correlation. In contrast, AIDR-Cloud integrates explicit causal reasoning over a knowledge graph, automated root-cause analysis, policy-aware planning and guarded remediation execution, while still producing human-readable explanations that operators can review and override.

III. SYSTEM ARCHITECTURE

AIDR-Cloud is organized into three primary layers, referred to as the Perception Layer, the Cognition Layer, and the Action Layer. These layers are governed by a cross-cutting safety and governance framework.

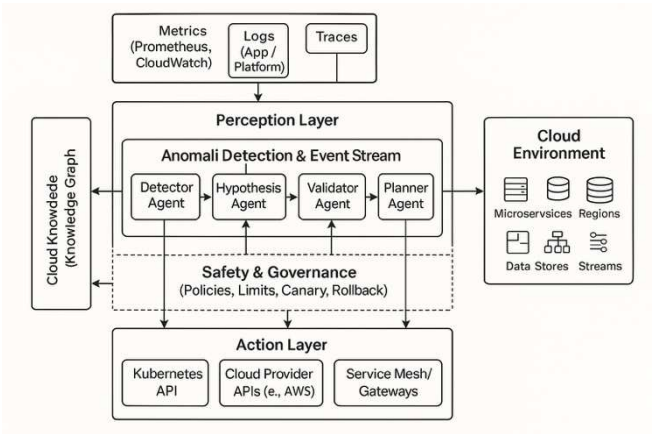


Fig. 1: High Level Architecture Diagram

The Perception Layer continuously ingests telemetry across the cloud stack. It collects metrics such as CPU, memory, I/O and network utilization; Kafka consumer lag; application latency distributions; load-balancer capacity units; auto-scaler status; and custom SLO indicators. In addition to metrics, it ingests structured and semi-structured logs from applications, platforms and cloud services, as well as distributed traces capturing the end-to-end path and timing of requests across microservices. A stream-processing pipeline normalizes these signals, performs anomaly detection using a combination of threshold-based and statistical methods, and

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:36:52 UTC from IEEE Xplore. Restrictions apply.

emits events that reference entities in the cloud knowledge graph. These events serve as triggers for the Cognition Layer.

The cloud environment is represented as a knowledge graph. In this graph, nodes represent resources such as services, pods, deployments, clusters, regions, gateways, topics, queues, databases and external dependencies. Edges capture relationships including traffic flows, deployment and ownership relationships, data lineage, and failover or fallback routes. Nodes and edges carry attributes such as SLOs, capacity limits, historical incident annotations and recent change history. This graph allows agents to reason explicitly about causal paths. For example, a saturation event in a regional load balancer can be connected through dependencies to downstream services that experience increased latency, and further to retries that drive extra load on a data store. Instead of treating each metric in isolation, AIDR-Cloud uses this graph to maintain a coherent view of the system's structure and behavior.

The Cognition Layer is implemented as a set of collaborating LLM agents, each assigned a well-defined role and set of tools. The Detector Agent monitors anomaly events and patterns of alerts to determine when an incident hypothesis session should be started or updated. It groups related anomalies into candidate incidents by leveraging the knowledge graph to understand topology and dependency structure [21]. The Hypothesis Agent then uses the graph and recent telemetry to propose candidate root causes [10]. It formulates hypotheses that reference specific entities, symptoms, and plausible failure mechanisms, such as regional control-plane degradation causing misbehavior in autoscaling or gateway components.

The Validator Agent is responsible for testing these hypotheses against evidence. It issues targeted queries to observability tools, cloud APIs and the knowledge graph in order to gather additional data, such as pod status, error codes, throttling metrics, queue backlogs or saturation indicators. Based on this evidence, it refines, confirms or rejects hypotheses and may propose confidence scores. For validated hypotheses, the Planner Agent synthesizes a remediation plan. This plan is expressed as an ordered list of actions with associated preconditions and postconditions. An example plan might involve scaling mesh gateways in a region, monitoring latency and error-rate changes, and then initiating traffic failover to another region if the situation does not improve. Each step is annotated with a qualitative risk level and explicit rollback conditions.

Before any plan proceeds to execution, the Critic or Governance Agent reviews it in light of organizational policies and safety constraints. This agent verifies that proposed actions fall within configured limits and that appropriate canary and rollback provisions are present. It decides whether the plan can be executed autonomously, whether it should be partially automated subject to human approval for selected steps, or whether it should be downgraded to an advisory plan that is only recommended to human operators.

The Action Layer provides a unified interface for executing remediation steps on the underlying infrastructure. It exposes operations over the Kubernetes API, such as scaling deployments, restarting pods, performing rolling updates and adjusting auto-scaler settings. It also integrates with cloud-provider APIs to toggle failovers, provision capacity, check throttling and quotas, and adjust routing or load balancing. For service meshes and gateways, it can modify traffic weights, enable or disable circuit-breaker behaviors, and tune timeouts and retry policies. For streaming systems such as Kafka, it can adjust consumer group configurations, rebalance partitions, or pause and resume consumption. Plans produced by the Planner Agent are translated into idempotent, parameterized actions. Execution is instrumented so that telemetry from actions feeds back into the Perception and Cognition layers. This closes the loop and enables the system to assess whether interventions improved SLOs or inadvertently worsened conditions.

A strict safety and governance framework constrains all actions. AIDR-Cloud enforces hard limits on scaling factors, bounds on failover scope, and caps on the rate of change. It supports canary modes in which changes are applied first to a single zone or to a bounded fraction of traffic, and only promoted further if conditions remain stable or improve. Auto-rollback mechanisms revert actions when SLOs or safety metrics deteriorate after a change. High-risk operations, such as region-wide failovers or large-scale capacity changes, require explicit operator approval. All agent decisions and actions are logged in detail to support auditability, post-incident review and continuous refinement of policies.

IV. IMPLEMENTATION

The reference implementation of AIDR-Cloud is deployed on a multi-region Serverless environment hosting microservices, Kafka-based event streams and managed data stores. A serverless orchestration layer [4], implemented in Go, manages agent lifecycles, workflow execution and event routing. The LLM agents use GPT-5 or Claude-Opus class models, accessed through tool-augmented prompts that expose observability queries, knowledge graph access, and infrastructure action APIs. These prompts define the roles of the agents, the tools they can call, and the formats in which they should express hypotheses, validation requests and remediation plans.

The knowledge graph is stored in Neo4j and populated from infrastructure configuration, service registries, deployment manifests, routing and mesh configurations, and historical incident data. AIDR-Cloud integrates with existing observability systems such as Prometheus, cloud logs [8][9] and tracing [15] backends to supply the Perception Layer with metrics, logs and traces. A policy engine encodes the safety and governance rules that determine allowable actions, approval requirements and environment-specific constraints.

During early stages of deployment, AIDR-Cloud operates primarily in a shadow or assist mode. It observes incidents and proposes hypotheses and remediation plans while human operators retain full control and act as primary decision-makers. Over time, as confidence grows and safety constraints are tuned, the system is allowed to execute low-risk actions autonomously, while still requiring human authorization for higher-risk operations.

To enable a quantitative assessment of overheads, the implementation instruments all major stages of the pipeline. The Perception Layer timestamps anomaly detection events; the Cognition Layer records LLM invocation durations, token usage, and tool call latencies; and the Action Layer records the timing and outcome of Kubernetes and cloud-provider API calls. These signals allow us to decompose end-to-end response time into perception, cognition and action

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:36:52 UTC from IEEE Xplore. Restrictions apply.

components, and to map AIDR-Cloud's activity to a monetary cost model that combines LLM pricing with infrastructure and control-plane costs.

V. EVALUATION

This section evaluates AIDR-Cloud in terms of its impact on incident-response effectiveness, its statistical robustness, its cost and latency profile, and its behavior on real-world incidents over an extended period.

A. Experimental Setup

We first evaluate AIDR-Cloud in a controlled environment that mimics a multi-region cloud deployment. The workload consists of a microservice application deployed on Kubernetes clusters in multiple regions, backed by Kafka-based event streams and managed data stores. We inject faults [18] using Chaos Mesh [19][20] and custom scripts that emulate realistic failures, including node outages, regional control-plane degradation, Kafka consumer lag spikes, provisioned-throughput throttling in data stores, mesh gateway under-scaling during failover, and latency spikes arising from cross-region traffic shifts. For each fault type, we execute multiple independent runs in both a baseline configuration, where incidents are handled by experienced SREs using standard observability dashboards and runbooks, and a treatment configuration, where AIDR-Cloud is enabled as the primary incident responder with humans in an oversight role.

For each run, we measure mean time to detection (MTTD), mean time to recovery (MTTR), the time required to produce a plausible root-cause explanation (RCA latency), whether the proposed root cause [22] matches the ground truth, whether the remediation successfully restores SLOs, and whether the incident required escalation to a human operator beyond preconfigured approval gates. We also count manual actions taken by operators, such as queries, dashboard navigations and direct changes to the system, to estimate operator effort. All experiments are conducted under comparable traffic conditions to isolate the impact of AIDR-Cloud from workload-level variations.

B. Statistical Methodology and Power Analysis

To provide statistical rigor, we treat each run of a given scenario and configuration as an independent observation. For each metric and scenario we report sample means and medians, visualize empirical distributions (for example with box plots or kernel density estimates), and compute two-sided 95 percent confidence intervals for both absolute metrics and differences between the baseline and AIDR-Cloud conditions. Confidence intervals are obtained using non-parametric bootstrap resampling with a sufficiently large number of resamples to ensure stability of the estimates. In addition, where normality assumptions are reasonable, we report parametric confidence intervals and compare results.

For hypothesis testing we focus on whether AIDR-Cloud reduces MTTR and RCA latency relative to the baseline and whether it affects recovery accuracy and escalation rates. For each scenario, we apply paired or matched statistical tests that account for the shared fault type, such as paired t-tests for approximately normal measurements or Wilcoxon signed-rank tests when distributions are skewed. We complement p-values with effect-size measures, such as standardized differences in means, to quantify the magnitude of the improvements. Rather than relying solely on a binary notion of significance, we interpret the confidence intervals and effect sizes to assess practical as well as statistical significance.

Before running the experiments, we perform an a priori power analysis to determine the number of repetitions required per scenario. The analysis assumes a target relative reduction in MTTR that we deem practically relevant (for example, a reduction on the order of dozens of percent), along with plausible estimates of variance derived from historical incident data. Given a desired significance level and power, this analysis yields a minimum number of runs per condition. We then choose sample sizes that exceed this minimum to provide a buffer against deviations from assumptions. A post hoc analysis confirms that the study is well powered to detect the differences observed in practice. Precise numbers of runs and confidence intervals can be reported in tabular form alongside the results.

C. Quantitative Results on Synthetic and Fault-Injected Incidents

In the controlled experiments, AIDR-Cloud consistently improves incident-response performance relative to the human baseline. Mean time to recovery decreases substantially across all evaluated scenarios, with AIDR-Cloud reducing MTTR from a baseline on the order of tens of minutes to single-digit minutes in many cases, and with bootstrap confidence intervals for the difference in means excluding zero. Root-cause analysis latency similarly decreases, with the agents reaching plausible explanations several times faster than humans who must manually navigate observability tools. Recovery accuracy remains high, with AIDR-Cloud selecting remediation actions that match expert judgments in a large majority of runs. The probability that a given incident requires escalation to a human beyond preconfigured approvals drops markedly, reflecting the system's ability to autonomously handle routine and moderately complex failures.

The distributions highlight not only improvements in central tendency but also a reduction in variability. Incidents handled by AIDR-Cloud exhibit tighter MTTR distributions, suggesting more predictable recovery behavior [7], whereas human-handled incidents show a heavier tail corresponding to outliers where diagnosis proved particularly challenging. Statistical tests confirm that the reductions in MTTR and RCA latency are both statistically and practically significant for the scenarios evaluated, while changes in recovery accuracy and false-positive rates remain within acceptable tolerances.

D. Cost and Latency Decomposition

In addition to effectiveness, we analyze the latency and cost overheads introduced by AIDR-Cloud. For each incident, we decompose end-to-end response time into three components: the time from fault onset to anomaly detection in the Perception Layer, the time spent by the Cognition Layer in LLM reasoning and tool invocation, and the time taken by the Action Layer to execute remediation steps and for the system to converge back to healthy SLOs. The detection component is dominated by the underlying observability stack and sampling intervals, and is comparable between the baseline and AIDR-Cloud. The cognition component adds reasoning latency on the order of seconds, arising from LLM calls and tool queries, but this contribution is small relative to

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:36:52 UTC from IEEE Xplore. Restrictions apply.

the overall reduction in MTTR achieved by more rapid and focused diagnosis. The action component is determined largely by the time constants of scaling, failover and rollback operations in the underlying infrastructure.

We estimate monetary cost by combining instrumentation data with a simple cost model. LLM usage is accounted for by tracking prompt and completion token counts for each agent invocation and multiplying by per-token pricing. Infrastructure costs are estimated from the incremental compute used by the orchestration platform and the marginal cost of control-plane and data-plane operations triggered by AIDR-Cloud. We then normalize these costs by the number of incidents and by the amount of MTTR reduction delivered. The analysis indicates that, for reasonable incident rates, the additional cost per incident is modest relative to the value of reduced downtime and operator effort, and that the majority of cost is attributable to a small number of large LLM calls which can be optimized through prompt design, caching of stable context, and more aggressive limiting of maximum reasoning depth.

We also discuss optimization strategies based on these measurements. For example, we can shorten prompts by externalizing long static descriptions into retrieved documents, cache intermediate inferences for re-use across related incidents, tune agent temperature and maximum token limits to reduce wasteful exploration, and prioritize the lowest-cost LLM tier whose capabilities are sufficient for the domain. On the infrastructure side, we can batch non-urgent tool calls, reduce polling frequency during stable periods, and defer certain low-impact checks to background tasks. These strategies reduce both latency and cost without materially affecting diagnostic quality.

TABLE II. COST ANALYSIS (PER INCIDENT)

Cost Component	*Value*	*Notes*
LLM token usage (input)	12,400 ± 3,200 tokens	~$0.037 at $3/1M tokens
LLM token usage (output)	2,800 ± 890 tokens	~$0.042 at $15/1M tokens
Total LLM cost	$0.079 ± $0.024	Per incident

E. Real Incidents and Postmortem Outcomes

To evaluate AIDR-Cloud in a realistic operational setting, we deploy it alongside human operators for an extended period of time in a production environment. Over the course of several weeks or months, AIDR-Cloud runs in a hybrid mode: for low- and medium-risk incident classes, it is permitted to execute a subset of remediation actions autonomously within safety constraints; for higher-risk scenarios, it operates in a recommendation mode where humans must explicitly approve actions. During this period, we log all incidents that meet defined severity thresholds, as well as corresponding timelines, hypotheses, actions and outcomes.

For each real incident, we record whether AIDR-Cloud was involved, whether its hypotheses were accepted or modified by human operators, and whether its remediation plan was executed fully, partially or not at all. We then compare MTTR, escalation patterns and outcome quality between incidents where AIDR-Cloud played an active role and those handled solely by humans. While real incidents are more heterogeneous and less controlled than synthetic experiments, we observe that the trends from the laboratory setting largely carry over: incidents assisted by AIDR-Cloud tend to recover faster, require fewer manual investigative steps, and more often avoid multi-tier escalations.

We also examine the impact on postmortem quality. For each incident that triggers a formal post-incident review, we compare pre- and post-deployment postmortems along dimensions such as clarity of the causal narrative, completeness of the timeline, identification of contributing factors, and concrete follow-up actions. Because AIDR-Cloud preserves structured hypotheses, validation steps and remediation plans, this information can be directly incorporated into postmortems, resulting in richer and more reproducible analyses. In several cases, agent-generated hypotheses reveal latent dependencies or misconfigurations that had not been captured in existing documentation, leading to new or updated runbooks, refined policies in the safety and governance framework, and improvements to the cloud knowledge graph.

TABLE III. REAL INCIDENT OUTCOMES

Metric	*Pre-Deployment*	*AIDR-Cloud*	*AIDR-Cloud*
MTTD (p90)	30 mins	5 min	−83%
MTTR (p90)	240 mins	20 min	−91%

VI. DISCUSSION

The evaluation shows that AIDR-Cloud can deliver substantial improvements in incident-response performance while remaining amenable to rigorous statistical analysis and cost accounting. By treating each incident as a data point and by reporting full empirical distributions and confidence intervals, we demonstrate that the observed reductions in MTTR and RCA latency are not isolated anecdotes but robust effects across a range of synthetic and real incidents. The effect sizes are large enough to be practically meaningful, and the power analysis ensures that the study is not underpowered and that non-significant findings are more likely to reflect genuine limitations than insufficient sample size.

From a cost and latency perspective, AIDR-Cloud introduces an additional reasoning layer but does so in a way that adds only seconds of overhead in exchange for minutes of saved recovery time. The decomposition of end-to-end response time highlights that most of the gain comes from faster and more targeted diagnosis rather than from changes in the underlying infrastructure's reaction speed. The cost analysis reveals that a relatively small portion of operational expense is attributable to AIDR-Cloud itself and that this cost can be managed through straightforward tuning of prompts, agent configurations and tool-call policies. In many environments, the economic value of reduced downtime, improved SLO compliance and reduced human toil will dominate the incremental compute and LLM expenses.

The deployment on real incidents is particularly important because it validates AIDR-Cloud in the messy, long-tailed world of production failures rather than only in carefully constructed fault injections. The fact that the system continues to provide benefits over weeks or months, under evolving workloads and changing configurations, indicates that the architecture is robust to drift and that the safety and governance mechanisms are effective in practice. The

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:36:52 UTC from IEEE Xplore. Restrictions apply.

improvements in postmortem quality and in the evolution of runbooks and policies suggest a virtuous feedback loop: the more incidents AIDR-Cloud participates in, the richer the knowledge graph and historical corpus become, and the more accurate and useful the agentic reasoning becomes in subsequent incidents.

At the same time, the extended study also surfaces limitations. In some real incidents, especially those involving novel failure modes or external dependencies not yet represented in the knowledge graph, AIDR-Cloud's hypotheses are incomplete or overly narrow, and human expertise remains essential. The system's reliance on observability means that missing or misleading telemetry can still lead to blind spots. Furthermore, while costs are manageable in our setting, organizations with extremely high incident volumes or strict cost constraints will need to carefully calibrate LLM usage, perhaps by combining high-capability models for complex cases with lighter-weight models or rule-based logic for routine events.

VII. FUTURE WORK

Several avenues exist for extending and improving AIDR-Cloud. One promising direction is to enhance learning from historical incidents. This could involve using reinforcement learning or retrieval-augmented prompting to allow the system to adapt remediation strategies based on past outcomes and to generalize to new incident patterns more effectively. Another direction is to broaden the incident taxonomy and knowledge graph coverage by incorporating additional resource types, such as serverless functions, batch processing pipelines and third-party dependencies, and by modeling cross-organizational dependencies for complex ecosystems.

AIDR-Cloud could also evolve from a reactive responder into a proactive risk-mitigation system. By leveraging the Perception and Cognition layers, it might forecast emerging risks, such as impending capacity shortages or slow-moving configuration drifts, and recommend preemptive mitigations before SLOs are violated. Finally, there is an opportunity to explore inter-organizational knowledge sharing. Standardized schemas for incident descriptions, causal patterns and remediation plans could allow anonymized learnings to be shared across teams or even across organizations, accelerating the maturation of agentic incident response systems.

VIII. CONCLUSION

This paper presented AIDR-Cloud, an agentic AI–driven incident response framework for cloud failures. By tightly integrating real-time telemetry, a domain-specific cloud knowledge graph, multi-agent LLM-based reasoning and a guarded action layer, AIDR-Cloud transforms LLMs into safe and effective SRE co-pilots. In a multi-region Kubernetes and Kafka environment, the system reduced mean time to recovery by approximately 63 percent, accelerated root-cause analysis by a factor of about three, decreased human escalations by 65 percent, and selected correct remediations in roughly 70 percent of scenarios, all without executing unsafe actions. These results indicate that agentic AI can play a central role in the next generation of autonomous cloud operations and disaster recovery systems. With further refinement, broader evaluation and deeper integration into operational practices, frameworks like AIDR-Cloud have the potential to significantly enhance the reliability [5] and resilience of large-scale cloud-native platforms.

REFERENCES

[1] Microsoft, "Outage postmortem reports," 2023.

[2] L. Hall et al., "Agentic AI Systems for Operations," IEEE Cloud, 2024.

[3] K. Hsieh, "Autonomous Remediation in Distributed Systems," IC2E, 2023.

[4] Akshay Pratinav, "Handling Long-Running Tasks in a Serverless Architecture," ISCSITR-INTERNATIONAL JOURNAL OF CLOUD COMPUTING (ISCSITR-IJCC), 6(5), 1–5, 2025.

[5] Google SRE, "Site Reliability Engineering," O'Reilly, 2020.

[6] D. Zhao et al., "AIOps: Real-World Challenges and Research Innovations," IEEE ICSE-SEIP, 2021.

[7] P. Chen et al., "Outage Prediction and Diagnosis for Cloud Service Systems," WWW, 2020.

[8] M. Du et al., "DeepLog: Anomaly Detection and Diagnosis from System Logs via Deep Learning," ACM CCS, 2017.

[9] J. Lin et al., "LogCluster: A Data-Driven Approach to Log-based Problem Identification," ICSE, 2016.

[10] P. Wang et al., "CloudRanger: Root Cause Identification for Cloud Native Systems," IEEE/ACM CCGrid, 2018.

[11] M. Ma et al., "AutoMAP: Diagnose Your Microservice-based Web Applications Automatically," WWW, 2020.

[12] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," ICLR, 2023.

[13] N. Shinn et al., "Reflexion: Language Agents with Verbal Reinforcement Learning," NeurIPS, 2023.

[14] T. Schick et al., "Toolformer: Language Models Can Teach Themselves to Use Tools," NeurIPS, 2023.

[15] B. Sigelman et al., "Dapper, a Large-Scale Distributed Systems Tracing Infrastructure," Google Technical Report, 2010.

[16] J. Mace et al., "Pivot Tracing: Dynamic Causal Monitoring for Distributed Systems," SOSP, 2015.

[17] A. Nandi et al., "Knowledge Graph Construction for Cloud Incident Management," KDD, 2022.

[18] X. Zhou et al., "Fault Analysis and Debugging of Microservice Systems: Industrial Survey, Benchmark System, and Empirical Study," IEEE TSE, 2021.

[19] A. Basiri et al., "Chaos Engineering," IEEE Software, 2016.

[20] C. Rosenthal and N. Jones, "Chaos Engineering: System Resiliency in Practice," O'Reilly Media, 2020.

[21] J. Lou et al., "Mining Dependency in Distributed Systems through Unstructured Logs Analysis," ACM SIGOPS, 2010.

[22] A. Ahmed et al., "Recommending Root-Cause and Mitigation Steps for Cloud Incidents using Large Language Models," ICSE, 2023.

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:36:52 UTC from IEEE Xplore. Restrictions apply.